

College Routine

Gamaliel Kalefe

26 mai 2026

Table des matières

1	Introduction	4
2	Problématique	4
3	Vision générale du produit	5
4	Objectifs du système	5
4.1	Réussite académique	5
4.2	Santé et récupération	6
4.3	Performance physique	6
5	Public cible	6
6	Principes scientifiques et pédagogiques	6
7	Principes fondamentaux du système	7
7.1	Objectifs clairs	7
7.2	Priorisation intelligente	7
7.3	Planification multiniveau	7
8	Modules principaux du système	8
8.1	Tableau de bord principal	8
8.2	Gestion académique	8
8.3	Plannification de sessions d'étude	8
8.3.1	Types de sessions possibles	9
8.4	Plannification de sessions d'entraînement physique	9
8.5	Moteur de santé et de récupératio	9
8.6	Système de recommandations intelligentes	9
8.6.1	Exemples de recommandations	10

8.7	Système anti-procrastination	10
8.8	Système de gestion du stress	10
9	Modèle énergétique	10
9.1	Énergie physique	11
9.2	Énergie cognitive	11
10	Données utilisées	11
11	Architecture générale du système	11
11.1	Frontend Web	12
11.2	Application iPhone	12
11.3	Backend intelligent	12
12	Contraintes et limites	12
13	Vision future	12
14	Analyse et conception du système	13
14.1	Nom du système	13
14.2	Acteurs du système	13
14.2.1	Acteur principal	13
14.2.2	Acteurs secondaires	13
14.3	Modèle conceptuel du domaine	13
14.4	Diagramme de cas d'utilisation	14
14.5	Cas d'utilisation détaillés	15
14.5.1	UC-01 : Générer un planning quotidien	15
14.5.2	UC-02 : Créer une session d'étude	16
14.5.3	UC-03 : Créer une session fitness	17
14.5.4	UC-04 : Recevoir une recommandation explicable	17
14.6	Diagramme de séquence système	18
14.6.1	Code PlantUML du SSD : Générer un planning quotidien	18
14.7	Diagrammes de séquence de conception	18
14.7.1	Générer un planning quotidien	18
14.7.2	Créer une session d'étude	20
14.7.3	Adapter une session fitness	21
14.8	Assignation des responsabilités	21
14.8.1	Contrôleur principal	22
14.8.2	Experts en information	22
14.8.3	Créateurs	22
14.8.4	Faible couplage	22
14.8.5	Forte cohésion	23

14.9 Patrons de conception utilisés	23
14.9.1 Patron Stratégie	23
14.9.2 Patron Façade	23
14.9.3 Patron Fabrique	23
14.9.4 Patron Observateur	24
14.9.5 Patron Commande	24
14.10 Diagramme d'activité	24
14.10.1 Code PlantUML du diagramme d'activité	24
14.11 Diagramme de classes de conception	25
14.11.1 Code PlantUML du diagramme de classes	25

1 Introduction

Le projet **College Routine** est une plateforme intelligente de gestion de performance humaine destinée principalement aux étudiants universitaires.

L'objectif du système est d'optimiser simultanément :

- la réussite académique ;
- la santé physique ;
- la récupération ;
- la gestion du stress ;
- l'organisation quotidienne ;
- le développement d'habitudes durables.

Le système agit comme un assistant intelligent capable de générer des recommandations personnalisées et des planifications dynamiques basées sur :

- les horaires de cours ;
- les plans de cours ;
- les échéances académiques ;
- les objectifs de l'utilisateur ;
- les données physiologiques provenant d'Apple Health ;
- les habitudes de vie ;
- des recherches scientifiques en neurosciences, productivité, sommeil et apprentissage.

2 Problématique

Les étudiants universitaires doivent constamment équilibrer :

- les cours ;
- les examens ;
- les projets ;
- les activités physiques ;
- le sommeil ;
- les responsabilités personnelles ;
- la santé mentale.

Les outils traditionnels tels que les calendriers et applications de tâches ne tiennent généralement pas compte :

- du niveau de fatigue ;

- du sommeil ;
- de la récupération ;
- de la charge cognitive ;
- des méthodes d'étude optimales ;
- ou de l'état psychologique de l'utilisateur.

Le projet vise donc à créer un système intelligent capable de transformer les données personnelles, académiques et physiologiques en recommandations concrètes et adaptatives.

3 Vision générale du produit

Le système doit fonctionner comme un planificateur capable de :

- comprendre l'état physique et mental de l'utilisateur ;
- analyser sa charge académique ;
- détecter les risques de surcharge ou de procrastination ;
- recommander des méthodes d'étude adaptées ;
- optimiser les périodes de concentration ;
- ajuster les entraînements physiques ;
- générer automatiquement un horaire quotidien intelligent.

Le système doit être :

- adaptatif ;
- explicable ;
- fondé scientifiquement ;
- flexible ;
- personnalisé.

4 Objectifs du système

4.1 Réussite académique

Le système doit permettre à l'utilisateur de :

- maximiser ses performances académiques ;
- améliorer sa compréhension profonde des matières ;
- réduire la procrastination ;
- optimiser ses méthodes d'étude ;

- planifier efficacement ses révisions ;
- maintenir un haut niveau de concentration ;
- atteindre un objectif GPA défini.

4.2 Santé et récupération

Le système doit permettre à l'utilisateur de :

- améliorer la qualité du sommeil ;
- éviter la fatigue chronique ;
- maintenir un bon équilibre de vie ;
- réduire le stress académique ;
- détecter les périodes de surcharge mentale.

4.3 Performance physique

Le système doit permettre à l'utilisateur de :

- atteindre des objectifs physiques précis ;
- optimiser les entraînements ;
- adapter les séances selon la récupération ;
- intégrer efficacement le sport dans l'horaire académique.

5 Public cible

Le système est principalement destiné :

- aux étudiants universitaires ;
- aux utilisateurs cherchant à optimiser leur performance cognitive ;
- aux utilisateurs souhaitant équilibrer études, santé et activités physiques.
- aux utilisateurs intéressés par le quantified self.

6 Principes scientifiques et pédagogiques

Le système s'appuie sur :

- des recherches scientifiques ;
- des conférences universitaires ;
- des ateliers pédagogiques ;
- des études sur l'apprentissage humain.

Les domaines utilisés incluent :

- neurosciences ;
- psychologie cognitive ;
- science du sommeil ;
- gestion du temps ;
- gestion du stress ;
- productivité ;
- science de l'apprentissage ;
- récupération physique ;
- méthodes d'étude.

7 Principes fondamentaux du système

7.1 Objectifs clairs

Le système doit encourager l'utilisateur à définir :

- des objectifs réalistes ;
- quantifiables ;
- limités dans le temps ;
- cohérents entre eux.

7.2 Priorisation intelligente

Le système doit utiliser une logique inspirée de la matrice d'Eisenhower afin de distinguer :

- les tâches urgentes ;
- les tâches importantes ;
- les tâches secondaires ;
- les tâches reportables.

7.3 Planification multiniveau

Le système doit permettre :

- une planification trimestrielle ;
- une planification hebdomadaire ;
- une planification quotidienne.

Le système doit conserver une certaine flexibilité afin d'adapter le planning aux imprévus et à l'état de l'utilisateur.

8 Modules principaux du système

8.1 Tableau de bord principal

Le tableau de bord principal doit afficher :

- le planning quotidien ;
- les tâches prioritaires ;
- les cours ;
- les séances d'étude ;
- les séances gym ;
- le niveau d'énergie ;
- les métriques de sommeil ;
- le niveau de récupération ;
- les recommandations principales du système.

8.2 Gestion académique

Le système doit permettre :

- l'importation des plans de cours ;
- l'analyse automatique des échéances ;
- la détection des périodes critiques ;
- l'estimation de charge de travail ;
- la planification des révisions ;
- le suivi de progression académique.

8.3 Plannification de sessions d'étude

Le système doit intégrer des sessions d'étude intelligentes similaires à des séances d'entraînement physique.

Chaque session peut inclure :

- un objectif ;
- une durée ;
- une intensité cognitive ;
- une méthode d'étude recommandée ;
- des pauses ;
- un résultat attendu.

8.3.1 Types de sessions possibles

- active recall ;
- résolution d'exercices ;
- lecture active ;
- deep work ;
- révision rapide ;
- création de mind maps ;
- préparation d'examens.

8.4 Plannification de sessions d'entraînement physique

Le système doit intégrer des séances physiques adaptées :

- aux objectifs physiques ;
- au niveau de récupération ;
- à la charge académique ;
- à l'énergie disponible.

8.5 Moteur de santé et de récupération

Le système doit utiliser les données provenant de :

- Apple Health ;
- Zepp ;
- d'autres sources compatibles.

Les données utilisées peuvent inclure :

- sommeil ;
- fréquence cardiaque ;
- activité physique ;
- calories ;
- récupération ;
- niveau d'activité.

8.6 Système de recommandations intelligentes

Le système doit générer des recommandations personnalisées basées sur :

- les données physiologiques ;
- la charge académique ;

- les objectifs ;
- les habitudes ;
- les recherches scientifiques utilisées.

8.6.1 Exemples de recommandations

- déplacer une séance de deep work ;
- réduire l'intensité d'un entraînement ;
- recommander une pause ;
- recommander une méthode d'étude ;
- prévenir une surcharge ;
- ajuster les horaires de sommeil.

8.7 Système anti-procrastination

Le système doit détecter les causes potentielles de procrastination :

- indécision ;
- ennui ;
- fatigue ;
- complexité ;
- manque de motivation.

Le système peut alors :

- segmenter automatiquement les tâches ;
- réduire la taille des objectifs ;
- proposer une méthode Pomodoro ;
- recommander une pause.

8.8 Système de gestion du stress

Le système doit intégrer :

- des recommandations de gestion du stress ;
- des conseils de préparation aux examens ;
- des stratégies de récupération mentale ;
- des recommandations de pauses.

9 Modèle énergétique

Le système doit distinguer :

9.1 Énergie physique

Influencée par :

- le sommeil ;
- les entraînements ;
- l'activité physique ;
- la récupération.

9.2 Énergie cognitive

Influencée par :

- la fatigue mentale ;
- la charge cognitive ;
- la concentration ;
- le stress ;
- la motivation.

10 Données utilisées

Le système peut utiliser :

- horaires de cours ;
- plans de cours ;
- échéanciers ;
- données Apple Health ;
- habitudes utilisateur ;
- tâches ;
- résultats académiques ;
- données de sommeil ;
- données d'activité physique ;
- sessions d'étude ;
- sessions gym.

11 Architecture générale du système

Le système comprendra principalement :

11.1 Frontend Web

- tableau de bord principal ;
- visualisations ;
- planification ;
- analytiques.

11.2 Application iPhone

- synchronisation Apple Health ;
- récupération des données santé ;
- envoi des données au backend.

11.3 Backend intelligent

- moteur de recommandations ;
- moteur de planification ;
- analyse de données ;
- stockage ;
- synchronisation temps réel.

12 Contraintes et limites

Le système doit :

- respecter la confidentialité des données santé ;
- permettre à l'utilisateur de contrôler ses données ;
- éviter les recommandations dangereuses ;
- éviter la surcharge excessive ;
- conserver une certaine flexibilité ;
- éviter la dépendance à une planification rigide.

13 Vision future

Les futures versions pourront intégrer :

- apprentissage automatique personnalisé ;
- prédiction de performance académique ;
- prédiction de fatigue ;

- détection avancée du burnout ;
- adaptation automatique des méthodes d'étude ;
- IA conversationnelle ;
- intégration calendrier avancée ;
- recommandations nutritionnelles ;
- visualisation des connaissances ;
- génération automatique de résumés et mind maps.

14 Analyse et conception du système

14.1 Nom du système

College Routine est une application intelligente qui aide l'utilisateur à organiser ses journées, ses cours, ses sessions d'étude, ses entraînements physiques, sa récupération et ses objectifs personnels.

14.2 Acteurs du système

14.2.1 Acteur principal

L'acteur principal est l'**utilisateur étudiant**. Il s'agit de la personne qui utilise l'application pour organiser sa vie académique, physique et personnelle.

14.2.2 Acteurs secondaires

Les acteurs secondaires sont :

- **Apple Health / Zepp** : fournit les données de santé, de sommeil, d'activité physique et de récupération ;
- **Google Calendar** : fournit ou reçoit les événements planifiés ;
- **Service IA** : participe à la génération des recommandations et à l'explication des décisions ;
- **Base scientifique** : contient les règles et sources scientifiques utilisées par le moteur de recommandations.

14.3 Modèle conceptuel du domaine

Le modèle conceptuel représente les concepts importants du domaine sans imposer de choix d'implémentation. Il permet d'identifier les objets métiers, leurs attributs principaux et leurs relations.

Le système repose sur un utilisateur qui possède des objectifs, suit des cours, importe des plans de cours, gère des échéances, planifie des tâches, réalise des sessions d'étude et des sessions fitness, puis reçoit des recommandations personnalisées.

Le système utilise également des données de santé afin d'estimer un état cognitif et un score de récupération. Ces informations influencent la génération du planning quotidien.

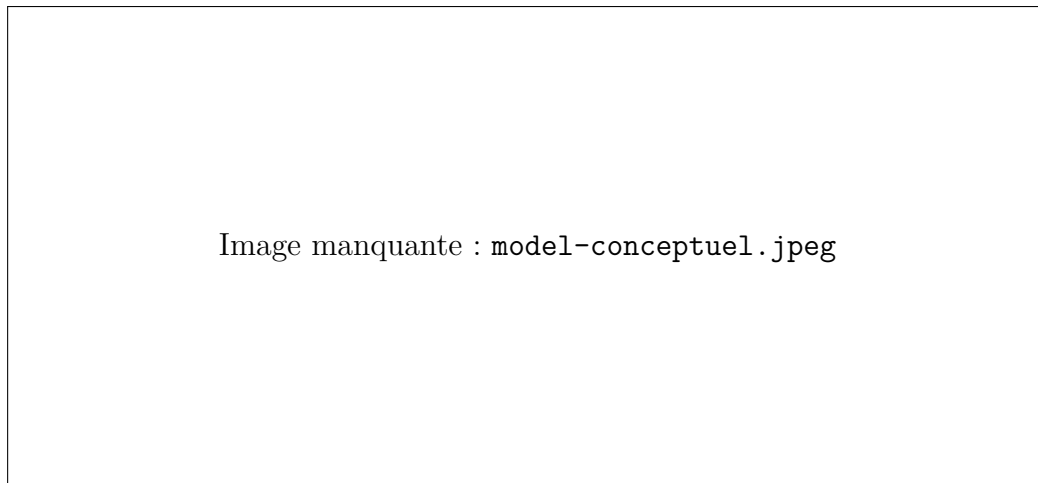


FIGURE 1 – Modèle conceptuel - College Routine

14.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales interactions entre les acteurs et le système.

Les cas d'utilisation principaux sont :

- configurer son profil ;
- définir ses objectifs ;
- importer un plan de cours ;
- gérer ses cours ;
- gérer ses échéances ;
- créer une session d'étude ;
- créer une session fitness ;
- synchroniser les données santé ;
- générer un planning quotidien ;
- recevoir des recommandations ;
- expliquer une recommandation ;
- suivre la progression académique ;
- suivre la progression physique.

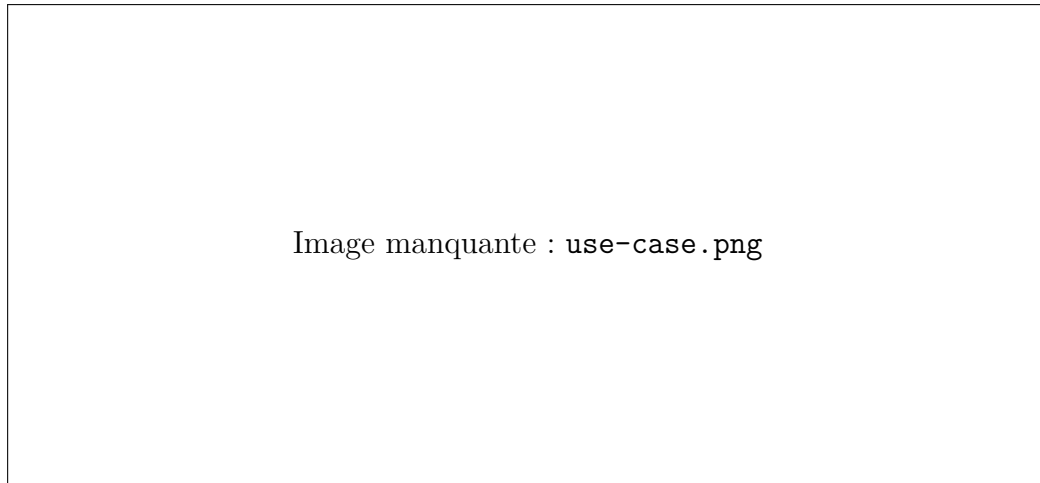


FIGURE 2 – Diagramme de cas d'utilisation - College Routine

14.5 Cas d'utilisation détaillés

14.5.1 UC-01 : Générer un planning quotidien

Acteur principal	Utilisateur étudiant.
Acteurs secondaires	Apple Health / Zepp, Google Calendar, Service IA, Base scientifique.
Préconditions	— L'utilisateur possède un profil configuré. — L'utilisateur a défini au moins un objectif académique, physique ou personnel. — Le système possède au moins une source de tâches, cours ou échéances.

TABLE 1 – Cas d'utilisation UC-01

Déclencheur L'utilisateur demande la génération du planning de la journée.

Scénario principal

1. L'utilisateur ouvre le dashboard.
2. Le système récupère les cours, tâches, échéances et contraintes du jour.
3. Le système synchronise les données santé disponibles.
4. Le système calcule l'état cognitif et le score de récupération.
5. Le système priorise les tâches selon leur urgence, leur importance et les objectifs.
6. Le système estime la charge académique du jour.

7. Le système sélectionne les sessions d'étude nécessaires.
8. Le système sélectionne ou ajuste les sessions fitness.
9. Le système génère les blocs de temps.
10. Le système ajoute les pauses, repas, périodes de récupération et sommeil.
11. Le système produit des recommandations explicables.
12. L'utilisateur consulte le planning.

Postconditions

- Un planning quotidien est créé.
- Les recommandations sont associées aux blocs de temps.
- Le planning peut être modifié par l'utilisateur.

Exceptions

- Si les données santé ne sont pas disponibles, le système utilise les données déclarées manuellement ou les valeurs précédentes.
- Si le planning est trop chargé, le système signale un risque de surcharge.
- Si deux activités sont incompatibles, le système propose une alternative.

14.5.2 UC-02 : Créer une session d'étude

Acteur principal Utilisateur étudiant.

Préconditions L'utilisateur possède au moins un cours.

Scénario principal

1. L'utilisateur sélectionne un cours.
2. Le système affiche les échéances et objectifs associés.
3. L'utilisateur choisit le type de session ou demande une recommandation.
4. Le système analyse la difficulté, l'urgence et l'état cognitif.
5. Le système recommande une méthode d'étude.
6. L'utilisateur confirme la session.
7. Le système ajoute la session au planning.

Méthodes possibles

- active recall ;
- practice testing ;
- spaced repetition ;
- mind map ;
- lecture active ;
- exercices ;
- Pomodoro.

14.5.3 UC-03 : Créer une session fitness

Acteur principal Utilisateur étudiant.

Préconditions L'utilisateur possède un objectif physique.

Scénario principal

1. L'utilisateur demande une séance gym.
2. Le système consulte les objectifs physiques.
3. Le système consulte le score de récupération.
4. Le système analyse la charge académique.
5. Le système propose une séance adaptée.
6. L'utilisateur confirme la séance.
7. Le système ajoute la séance au planning.

Exceptions Si la récupération est faible, le système recommande une séance légère ou une récupération active.

14.5.4 UC-04 : Recevoir une recommandation explicable

Acteur principal Utilisateur étudiant.

Scénario principal

1. Le système détecte une situation pertinente.
2. Le système applique les règles scientifiques.
3. Le système génère une recommandation.
4. Le système associe une explication claire à la recommandation.
5. L'utilisateur consulte la recommandation.
6. L'utilisateur peut accepter, ignorer ou modifier la recommandation.

14.6 Diagramme de séquence système

Le diagramme de séquence système présente les interactions entre l'acteur et le système, sans détailler les objets internes.

14.6.1 Code PlantUML du SSD : Générer un planning quotidien

```
@startuml
title SSD - Générer un planning quotidien - College Routine

actor "Utilisateur étudiant" as User
participant "Système College Routine" as System

User -> System : ouvrirDashboard()
System --> User : afficherRésuméJournee()

User -> System : demanderGenerationPlanning(date)
System -> System : récupérerCoursEtEcheances(date)
System -> System : récupérerTaches(date)
System -> System : synchroniserDonnéesSanté()
System -> System : calculerEtatCognitif()
System -> System : calculerScoreRecuperation()
System -> System : prioriserActivites()
System -> System : générerBlocsTemps()
System -> System : générerRecommandations()
System --> User : afficherPlanningQuotidien(planning, recommandations)

User -> System : confirmerPlanning()
System -> System : sauvegarderPlanning()
System --> User : afficherConfirmation()

@enduml
```

14.7 Diagrammes de séquence de conception

Les diagrammes de séquence de conception montrent comment les objets internes collaborent pour réaliser un cas d'utilisation.

14.7.1 Générer un planning quotidien

```
@startuml
title Diagramme de séquence - Générer un planning quotidien - College Routine
```

```

actor "Utilisateur" as User
boundary "DashboardView" as View
control "PlanningFacade" as Facade
control "AcademicService" as Academic
control "HealthService" as Health
control "PlanningEngine" as Planner
control "RecommendationEngine" as Reco
entity "PlanningContext" as Context
entity "DailyPlan" as Plan

User -> View : cliquerGenererPlanning(date)
View -> Facade : generateDailyPlan(userId, date)

Facade -> Academic : getAcademicData(userId, date)
Academic --> Facade : courses, tasks, deadlines

Facade -> Health : syncHealthData(userId)
Health --> Facade : healthData, recoveryScore, cognitiveState

Facade -> Context : create(user, courses, tasks, deadlines, healthData)
Context --> Facade : context

Facade -> Planner : generateDailyPlan(context)
Planner -> Planner : prioritizeTasks()
Planner -> Planner : allocateStudySessions()
Planner -> Planner : allocateFitnessSessions()
Planner -> Planner : insertBreaksAndRecovery()
Planner --> Facade : dailyPlan

Facade -> Reco : generateRecommendations(context, dailyPlan)
Reco -> Reco : applyScientificRules()
Reco -> Reco : generateExplanations()
Reco --> Facade : recommendations

Facade -> Plan : attachRecommendations(recommendations)
Plan --> Facade : updatedPlan

Facade --> View : dailyPlan
View --> User : afficherPlanning(dailyPlan)

```

@enduml

14.7.2 Créer une session d'étude

@startuml

title Diagramme de séquence - Créer une session d'étude - College Routine

actor "Utilisateur" as User
boundary "StudySessionView" as View
control "PlanningFacade" as Facade
control "AcademicService" as Academic
control "HealthService" as Health
control "RecommendationEngine" as Reco
control "StudySessionFactory" as Factory
entity "StudySession" as Session
entity "Course" as Course

User -> View : sélectionnerCours(courseId)
View -> Facade : getCourseContext(courseId)

Facade -> Academic : getCourse(courseId)
Academic --> Facade : course

Facade -> Health : getLatestCognitiveState(userId)
Health --> Facade : cognitiveState

Facade -> Reco : recommendStudyStrategy(course, cognitiveState)
Reco --> Facade : studyStrategy

Facade -> Factory : createStudySession(course, studyStrategy, cognitiveState)
Factory --> Facade : studySession

Facade --> View : afficherSessionProposée(studySession)
User -> View : confirmerSession()
View -> Facade : saveSession(studySession)
Facade --> View : confirmation()

@enduml

14.7.3 Adapter une session fitness

@startuml

title Diagramme de séquence - Adapter une séance fitness - College Routine

```
actor "Utilisateur" as User
boundary "FitnessView" as View
control "PlanningFacade" as Facade
control "HealthService" as Health
control "FitnessService" as Fitness
entity "RecoveryScore" as Recovery
entity "FitnessSession" as Session
```

```
User -> View : demanderSeanceFitness()
```

```
View -> Facade : createFitnessSession(userId)
```

```
Facade -> Health : getLatestRecoveryScore(userId)
```

```
Health --> Facade : recoveryScore
```

```
Facade -> Fitness : createFitnessSession(goal, recoveryScore)
```

```
Fitness -> Fitness : choisirIntensité()
```

```
Fitness -> Fitness : choisirTypeEntrainement()
```

```
Fitness --> Facade : fitnessSession
```

```
Facade --> View : afficherSeanceProposée(fitnessSession)
```

```
alt récupération faible
```

```
    View --> User : proposerSéanceLégèreOuRepos()
```

```
else récupération normale
```

```
    View --> User : proposerSéanceNormale()
```

```
end
```

@enduml

14.8 Assignment des responsabilités

L'assignation des responsabilités s'appuie sur les principes de faible couplage, forte cohésion et expert en information.

14.8.1 Contrôleur principal

La classe `PlanningFacade` agit comme contrôleur principal pour les cas d'utilisation liés au planning.

Ses responsabilités sont :

- recevoir les demandes provenant du dashboard ;
- orchestrer les services internes ;
- masquer la complexité du système ;
- retourner un planning complet à l'interface.

14.8.2 Experts en information

Les classes suivantes sont des experts en information :

- `Course` ;
- `Deadline` ;
- `Task` ;
- `HealthData` ;
- `RecoveryScore` ;
- `CognitiveState` ;
- `DailyPlan`.

Elles possèdent les informations nécessaires pour répondre aux questions du domaine.

14.8.3 Créateurs

La classe `PlanningEngine` est responsable de créer les objets `DailyPlan` et `TimeBlock`.

Les fabriques spécialisées sont responsables de créer les différents types de sessions :

- `StudySessionFactory` ;
- `FitnessSessionFactory` ;
- `RecoverySessionFactory`.

14.8.4 Faible couplage

Le système sépare les responsabilités dans plusieurs services :

- `AcademicService` ;
- `HealthService` ;
- `FitnessService` ;
- `RecommendationEngine` ;
- `PlanningEngine`.

Chaque service est responsable d'un domaine précis.

14.8.5 Forte cohésion

Chaque module du système possède une responsabilité claire :

- le module académique gère les cours, plans de cours, tâches et échéances ;
- le module santé gère les données physiologiques ;
- le module planning gère l'horaire quotidien ;
- le module sessions gère les sessions d'étude, fitness et récupération ;
- le module recommandations gère les règles scientifiques et les suggestions personnalisées.

14.9 Patrons de conception utilisés

14.9.1 Patron Stratégie

Le patron Stratégie est utilisé pour représenter les différentes méthodes d'étude.

Exemples :

- `ActiveRecallStrategy` ;
- `SpacedRepetitionStrategy` ;
- `PracticeTestingStrategy` ;
- `MindMapStrategy` ;
- `PomodoroStrategy`.

Ce patron permet de changer dynamiquement la méthode d'étude recommandée selon le cours, la fatigue, l'urgence ou la proximité d'un examen.

14.9.2 Patron Façade

Le patron Façade est utilisé avec `PlanningFacade`. Cette classe offre un point d'entrée simple pour l'interface utilisateur tout en masquant la complexité des services internes.

14.9.3 Patron Fabrique

Le patron Fabrique est utilisé pour créer les différents types de sessions :

- session d'étude ;
- session fitness ;
- session de récupération.

14.9.4 Patron Observateur

Le patron Observateur peut être utilisé pour notifier automatiquement les vues lorsque le planning change.

Exemples de vues observatrices :

- DashboardView;
- CalendarView;
- RecommendationView.

14.9.5 Patron Commande

Le patron Commande peut être utilisé pour rendre certaines actions annulables :

- créer une tâche;
- déplacer un bloc de temps;
- générer un planning;
- compléter une session.

14.10 Diagramme d'activité

Le diagramme d'activité suivant décrit le processus général de génération du planning quotidien.

14.10.1 Code PlantUML du diagramme d'activité

```
@startuml
title Diagramme d'activité - Génération du planning quotidien - College Routine

start

:Charger profil utilisateur;
:Charger objectifs;
:Charger cours, tâches et échéances;
:Synchroniser données santé;

:Calculer score de récupération;
:Calculer état cognitif;
:Estimer charge académique;

if (Surcharge détectée ?) then (oui)
    :Réduire blocs non critiques;
    :Ajouter pauses;
endif
```

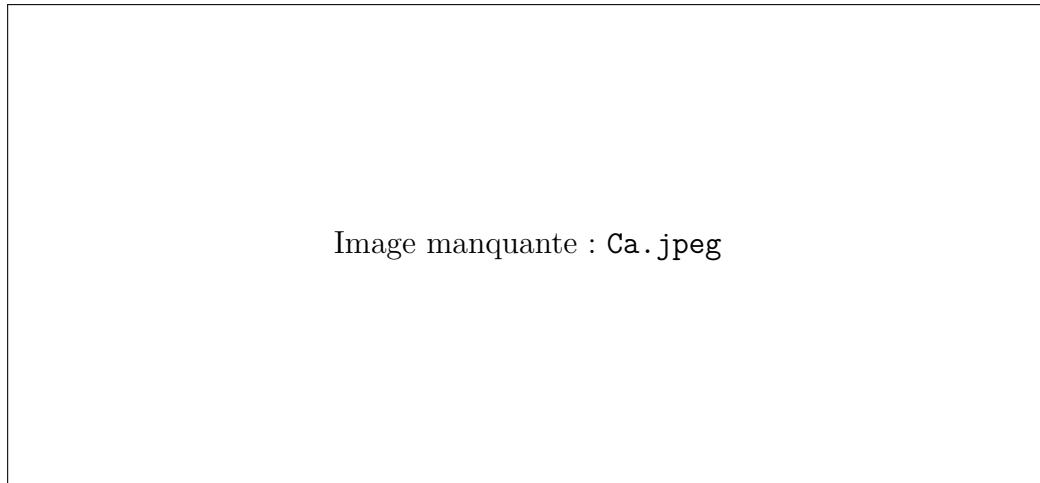



FIGURE 3 – Diagramme de classes de conception

```

    :Générer alerte de surcharge;
else (non)
    :Continuer planification normale;
endif

:Prioriser les tâches;
:Choisir sessions d'étude;
:Choisir session fitness;
:Créer blocs de temps;
:Ajouter repas, pauses et récupération;

:Appliquer règles scientifiques;
:Générer recommandations;
:Afficher planning;

stop

@enduml

```

14.11 Diagramme de classes de conception

Le diagramme de classes de conception présente une première architecture logicielle de **College Routine**. Il regroupe les classes selon leurs responsabilités principales.

14.11.1 Code PlantUML du diagramme de classes